LEXICAL ANALYZER

Introduction

A console-based lexical analyser in C which breaks the source code text into lexemes, matches lexemes with corresponding tokens and displays the token-lexeme pair.

Concepts used

String handling

File I/O

Character handling

Pointers

File-handling

FILES

1. main.c – Collects the input ".c" file through command line arguments, validates the file name, if file name is missing, a usage message is printed.
2. lexer.c-opens the file, parses the file character by character, classifies into appropriate tokens by calling necessary functions like, isKeyword(), isOperator(), isBracket(), isIntFloatConstant(), isCharStrConstant().

FEATURES

I. Displays the following lexemes and matching token names.

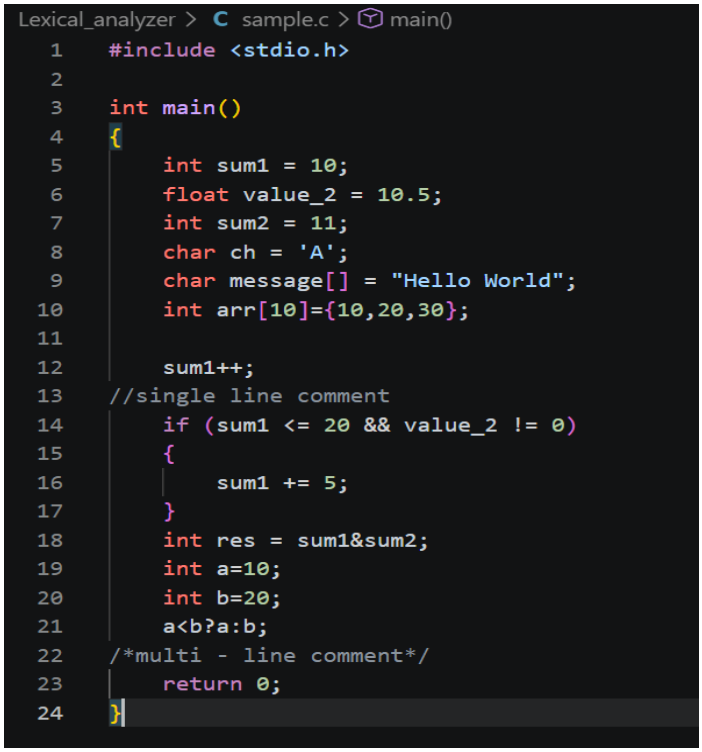
    1. KEYWORD
    2. IDENTIFIER
    3. INTEGER_CONSTANT
    4. FLOAT_CONSTANT
    5. CHAR_CONSTANT

6. STRING_CONSTANT
7. ARITHMETIC_OPERATOR
8. ASSIGNMENT_OPERATOR
9. RELATIONAL_OPERATOR
10. LOGICAL_OPERATOR
11. BITWISE_OPERATOR
12. INCREMENT_DECREMENT_OPERATOR
13. MEMBER_ACCESS_OPERATOR
14. OPEN_PARENTHESIS
15. CLOSE_PARENTHESIS
16. OPEN_CURLY_BRACE
17. CLOSE_CURLY_BRACE
18. OPEN_SQUARE_BRACKET
19. CLOSE_SQUARE_BRACKET
20. PREPROCESSOR_DIRECTIVE
21. SEMICOLON
22. COMMA
23. TERNARY_OPERATOR

II. Skips single and multiline comments

SAMPLE OUTPUT

Input file:



Output:

```
shreya@LAPTOP-2Q70AT88:~/cprograms/Lexical_analyzer$ ./lexical_analyzer sample.c
Opening sample.c....
Parsing sample.c....

PREPROCESSOR_DIRECTIVE        : #include <stdio.h>
KEYWORD                       : int
IDENTIFIER                    : main
OPEN_PARENTHESIS              : (
CLOSE_PARENTHESIS             : )
OPEN_CURLY_BRACE              : {
KEYWORD                       : int
IDENTIFIER                    : sum1
ASSIGNMENT_OPERATOR           : =
INTEGER CONSTANT              : 10
SEMICOLON                     : ;
KEYWORD                       : float
IDENTIFIER                    : value_2
ASSIGNMENT_OPERATOR           : =
FLOAT CONSTANT                : 10.5
SEMICOLON                     : ;
KEYWORD                       : int
IDENTIFIER                    : sum2
ASSIGNMENT_OPERATOR           : =
INTEGER CONSTANT              : 11
SEMICOLON                     : ;
KEYWORD                       : char
IDENTIFIER                    : ch
ASSIGNMENT_OPERATOR           : =
CHAR_CONSTANT                 : 'A'
SEMICOLON                     : ;
KEYWORD                       : char
IDENTIFIER                    : message
OPEN_SQUARE_BRACKET           : [
CLOSE_SQUARE_BRACKET          : ]
ASSIGNMENT_OPERATOR           : =
STRING_CONSTANT               : "Hello World"
SEMICOLON                     : ;
KEYWORD                       : int
```

```
IDENTIFIER                    : arr
OPEN_SQUARE_BRACKET           : [
INTEGER_CONSTANT              : 10
CLOSE_SQUARE_BRACKET          : ]
ASSIGNMENT_OPERATOR           : =
OPEN_CURLY_BRACE              : {
INTEGER_CONSTANT              : 10
COMMA                         : ,
INTEGER_CONSTANT              : 20
COMMA                         : ,
INTEGER_CONSTANT              : 30
CLOSE_CURLY_BRACE             : }
SEMICOLON                     : ;
IDENTIFIER                    : sum1
INCREMENT_DECREMENT_OPERATOR  : ++
SEMICOLON                     : ;
KEYWORD                       : if
OPEN_PARENTHESIS              : (
IDENTIFIER                    : sum1
RELATIONAL_OPERATOR           : <=
INTEGER_CONSTANT              : 20
LOGICAL_OPERATOR              : &&
IDENTIFIER                    : value_2
RELATIONAL_OPERATOR           : !=
INTEGER_CONSTANT              : 0
CLOSE_PARENTHESIS             : )
OPEN_CURLY_BRACE              : {
IDENTIFIER                    : sum1
ASSIGNMENT_OPERATOR           : +=
INTEGER_CONSTANT              : 5
SEMICOLON                     : ;
CLOSE_CURLY_BRACE             : }
KEYWORD                       : int
IDENTIFIER                    : res
ASSIGNMENT_OPERATOR           : =
IDENTIFIER                    : sum1
BITWISE_OPERATOR              : &
```

```
IDENTIFIER                    : sum2
SEMICOLON                     : ;
KEYWORD                       : int
IDENTIFIER                    : a
ASSIGNMENT_OPERATOR           : =
INTEGER_CONSTANT              : 10
SEMICOLON                     : ;
KEYWORD                       : int
IDENTIFIER                    : b
ASSIGNMENT_OPERATOR           : =
INTEGER_CONSTANT              : 20
SEMICOLON                     : ;
IDENTIFIER                    : a
RELATIONAL_OPERATOR           : <
IDENTIFIER                    : b
TERNARY_OPERATOR              : ?
IDENTIFIER                    : a
TERNARY_OPERATOR              : :
IDENTIFIER                    : b
SEMICOLON                     : ;
KEYWORD                       : return
INTEGER_CONSTANT              : 0
SEMICOLON                     : ;
CLOSE_CURLY_BRACE             : }

Parsing completed.
```